

AI Health Agent — Project Summary

Oct 3, 2026 · @DCoder

AI Health Agent is a full-stack conversational AI product for diet and exercise tracking, built around Indian food and eating habits. Instead of a logging form, a tool-calling agent (Google Gemini, on a FastAPI/Python backend) decides when to save, edit or ask for confirmation, remembers each user's patterns over time, and generates AI-synthesized weekly and monthly behavioural reports.

Product & Design Decisions

The core product decision: a conversational agent, not a form. The user describes what they ate or did in plain language; the agent decides intent and acts, rather than every message mapping to a fixed input field.

Confirmation tiers — not every message gets the same scrutiny:

Confidence	Behaviour
High	Save immediately, confirm in the reply
Medium	Save, but flag the estimate for the user to correct
Low	Ask a clarifying question before saving anything

Corrections ("actually 3 rotis") and undo ("undo that") are handled conversationally within the session, not as separate UI actions.

Edit and view rules:

Period	Allowed
Today, before midnight	Log, edit, delete, view
Yesterday and earlier	View only, locked forever

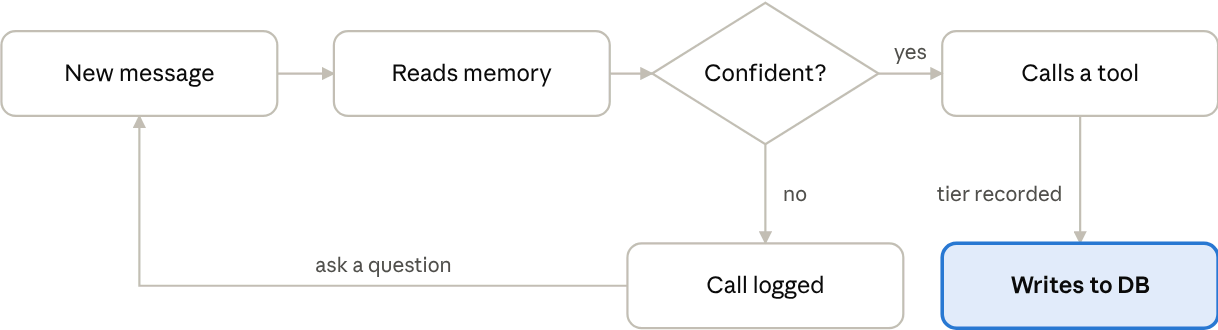
Cutoff is strict midnight (Option A, no grace period), evaluated against the user's own stored timezone — never server/UTC time. This keeps report numbers from silently changing after the fact and simplifies the agent's write-permission check to one date comparison.

Session model: one session per calendar day (Option C). Today’s session is the active chat; past days render as read-only summary views. Long-term memory (UserMemory) carries facts and patterns across that daily reset.

System Architecture

Two separate orchestration loops run on this system: the live chat agent, fired per message, and a scheduled pipeline (midnight cron) that runs end-of-day memory summarization and weekly/monthly report generation.

Confidence decides whether the agent asks or just saves



live agent loop, one decision point

Memory makes the confidence check smarter over time: early on the agent asks often, but as UserMemory fills in with usual meals and patterns, it can infer more from less and ask fewer clarifying questions.

Memory System

UserMemory is one row per user, continuously updated, never replaced — a living profile the agent reads before every reply, not a log of events.

	In-conversation update	End-of-day summarization
Trigger	Mid-message, same turn the agent is already answering	Once, by the midnight job, after the day locks
Input	One message	The full day's messages + that day's DailySummary

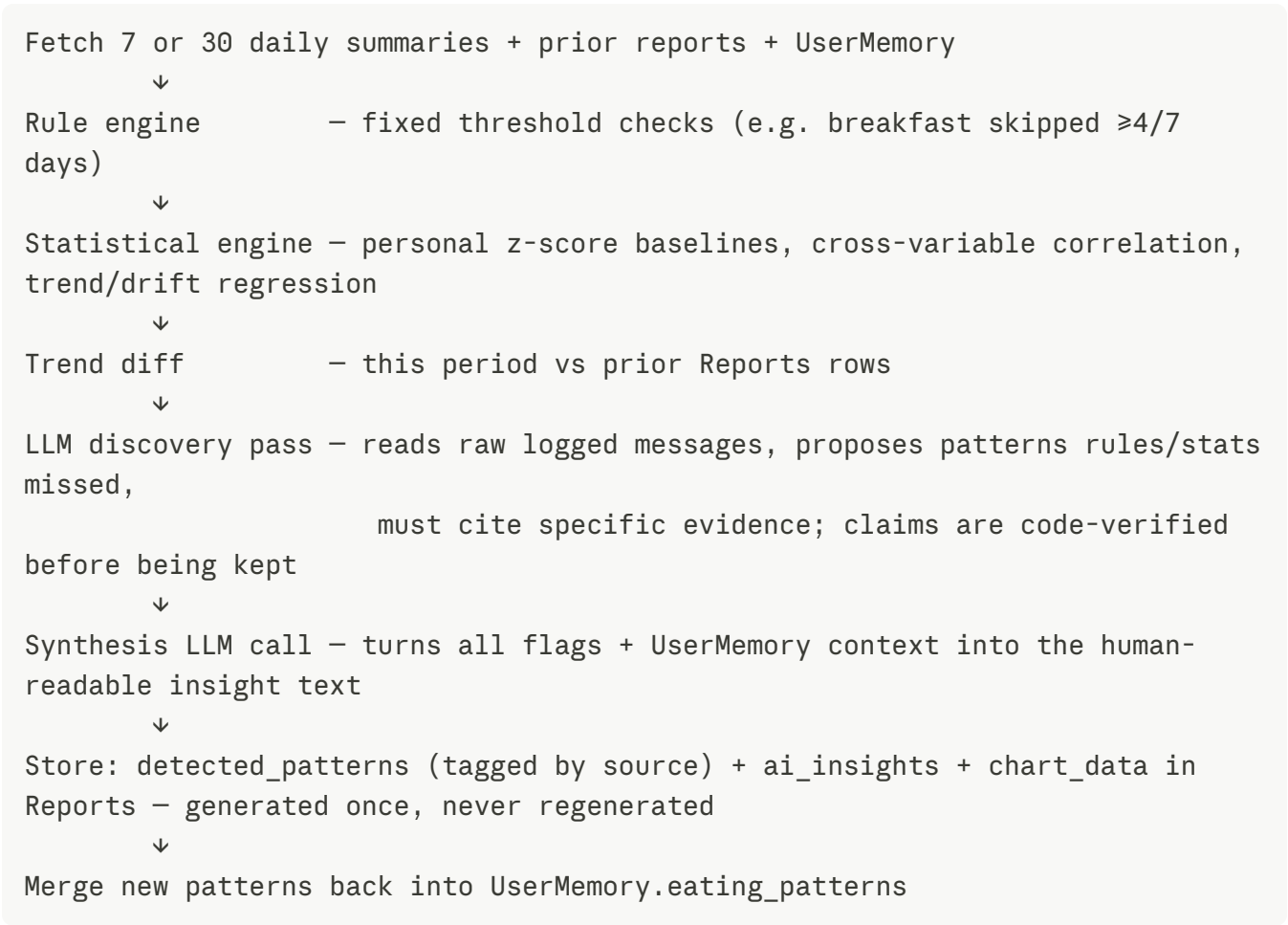
	In-conversation update	End-of-day summarization
Looks for	An explicitly stated, permanent fact	An implicit pattern across the day's data
Decision rule	"Would this still be true next week?"	Repetition threshold checked in code (e.g. ≥ 3 of 7 days) before writing
Writes to	<code>food_preferences</code> , <code>usual_meals</code>	<code>eating_patterns</code> , <code>typical_calories</code>
Example	"I'm vegetarian" → written immediately	3rd skipped breakfast this week → flag only written after the 7-day count passes

Both are needed: an allergy mentioned once would never surface from a same-day pattern check, and a creeping behavioural habit is never something a user states outright — it's only visible by looking back across days.

The in-conversation writer uses a structured tool (`update_user_memory`) with a fixed field enum, so the agent classifies into known buckets rather than inventing new memory structure. The end-of-day summarizer is a separate, dedicated LLM call — the user never sees it — that proposes candidate facts and observations; a code-level check then decides what's actually permanent or frequent enough to persist.

Pattern Detection & Reports

Pattern detection runs in three layers, each suited to a different kind of data — not a single hardcoded rule set, which personalises nothing and can't use the unstructured parts of a user's data.



The LLM is never asked to discover patterns from raw numbers alone — numeric patterns come from rules and statistics, and the LLM's two jobs are reading unstructured text for what numbers can't show, and writing the final narrative from facts it's handed. Reports are triggered by the same midnight/weekly/monthly cron that also runs end-of-day summarization (see Deployment & Background Jobs).

Data Model

Table	Holds
Users	Profile, goals, timezone, health conditions
FoodLogs	Raw message (always kept), meal type, confidence score
FoodItems	Parsed items, macros, nutrients per log
ExerciseLogs	Exercise type, duration, calories burned

Table	Holds
DailySummaries	Aggregated daily totals — reports read these, not raw logs
Sessions	One row per day; <code>memory_summarized</code> flag for safe retries
ConversationMessages	Full chat history, tool calls and parameters recorded
UserMemory	One row per user — long-term patterns and preferences
Reports	Weekly/monthly; <code>detected_patterns</code> (JSON) + <code>ai_insights</code> (text), stored forever
LLMMonitoringLogs	Tokens, cost, latency, confidence, success per AI call

Two decisions shaped this: the raw user message is always stored, so a parsing mistake can be reprocessed later with a better model without losing history; and generated report text is stored once and never regenerated, since LLM output is non-deterministic and re-running it on every view would both cost money and show the user a different report each time.

Tech Stack

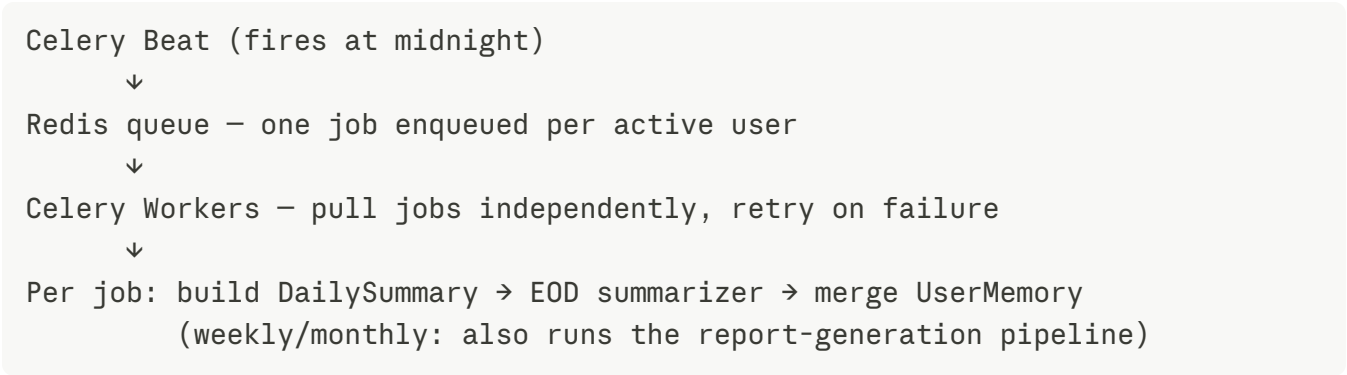
Layer	Choice
Frontend	Next.js, TypeScript, Tailwind CSS, Recharts, deployed on Vercel
Backend	FastAPI (Python) — switched from Node/Fastify since the AI/data ecosystem (Google's Generative AI SDK, pandas, numpy, pgvector) is Python-first
Database	PostgreSQL + an ORM (Prisma/SQLAlchemy)
AI / agent layer	Google Gemini, tool use (function calling) for all DB writes, confidence scoring, conversation + long-term memory

Layer	Choice
Background jobs	Celery (workers) + Celery Beat (scheduler) + Redis (queue)
Observability	Langfuse (tracing, cost/latency, evaluation), Sentry (errors)
Infra / deploy	Docker, Docker Compose (local), GitHub Actions (CI/CD), Railway (hosting)

The FastAPI switch and the Celery/Redis job layer were both deliberate trade-offs against a “pure cloud-native” alternative (AWS Lambda/SQS/EventBridge): more infra depth is a stronger resume story in theory, but risks months going into Terraform and IAM before the agent itself is proven to work well. The call was to build and validate the agent locally first, on boring/fast infrastructure, and treat a cloud-native rebuild as a separate, later phase if still wanted. The LLM vendor was also switched, from Anthropic Claude to Google Gemini, specifically for its free API tier: it removes per-call cost during development, at the cost of a less mature function-calling ecosystem and thinner first-party observability integrations than Claude currently has, worth revisiting once usage would actually hit the free tier’s limits.

Deployment & Background Jobs

Two kinds of work need to run outside the normal request/response cycle: a midnight trigger, and potentially hundreds of per-user LLM calls that shouldn’t block each other or the API.



Job-per-user matters for fault isolation — one user’s failed LLM call or malformed response can’t stall or crash the batch for everyone else. The `Sessions.memory_summarized` flag makes each job idempotent, so a retried or re-run job

never double-counts an observation toward the 7-day pattern threshold.

Deployed on Railway: the same container image for the API and the worker process, differing only in start command, plus a Redis instance for the queue.

Observability, Evaluation & Traceability

Pillar	Answers	Tooling
Observability	Is the system healthy right now?	Langfuse traces (nested per user turn) + LLMMonitoringLogs + Sentry
Evaluation	Is a prompt change actually better or worse?	Golden dataset + scoring functions, run as Langfuse experiments per prompt version
Traceability	Given this output, can I reconstruct exactly how it was produced?	<code>trace_id</code> + <code>prompt_version</code> stamped on every log row, linked from the originating table row

Observability — every LLM call is wrapped (`@observe()`) via the Langfuse SDK, using its native Google GenAI SDK wrapper, or an OpenTelemetry bridge where native support still lags Anthropic's), nesting intent classification, the model call, and the resulting tool call under one trace per user message.

Evaluation — a fixed set of roughly 20–30 real logged messages with known-correct outputs, scored per operation (numeric tolerance for food parsing, exact-match for tool selection, LLM-as-judge for report quality). Every prompt change is run against this set before being considered done, compared across prompt versions rather than eyeballed.

Traceability — `langfuse_trace_id` stored as a column on FoodLog, ConversationMessages and Reports, so any stored output is one click from its exact input, prompt version, model response and confidence score.

Decision: start with Langfuse Cloud rather than self-hosting — self-hosting needs its own ClickHouse + S3/MinIO stack, which would contradict the "keep infra simple, ship the agent first" call made for the rest of the system.

Open Decisions / Next Steps

- ☐ Wire the first Claude call (food parsing) through the Langfuse-wrapped client

- ☐ Build the golden dataset for food-parsing evaluation
- ☐ Decide whether weekly-report `detected_patterns` auto-merge into UserMemory, or require a review step first
- ☐ Revisit cloud-native deployment (EventBridge/SQS/Fargate) as a later, separate phase once the agent works end-to-end
- ☐ Finalize the build roadmap sequencing now that jobs and observability are designed in